

Kartik Maihuria
24116042

Automated Difficulty Estimation for Programming Problems using Natural Language Processing and ML Model Training

In the world of competitive programming (platforms like Codeforces, LeetCode, AtCoder), thousands of new problems are created regularly. Currently, assigning a difficulty level (e.g., "Easy," "Medium," "Hard") or a specific rating score (e.g., 800, 1500) to these problems is a largely manual process. Our Task is to develop an intelligent system ("AutoJudge") that can automatically predict the difficulty level and rating score of a programming problem solely based on its textual description.

Dataset Used:-

We are using a dataset (problems_data.json) which is then converted into problems.csv dataset suggested to use in the document.

.Data Fields:

Input Features (Text): description, input_description, output_description, title.

Target Labels:

problem_class: Categorical (Easy, Medium, Hard).

problem_score: Numerical (e.g., 4,6,8,...).

Field Name	Type	Description	Usage in Project
title	Text	The name of the problem (e.g., "Watermelon").	Metadata (Not used for training).
description	Text	The main storyline and logical requirements of the problem.	Primary Input Feature.
input_description	Text	Details about input constraints (e.g., "Integer N where $1 \leq N \leq 100$ ").	Combined into full_text.
output_description	Text	Details about expected output format.	Combined into full_text.

<code>problem_score</code>	Number	The specific rating (e.g., 4,7,10).	Target for Regression Model.
<code>problem_class</code>	String	The difficulty category (Easy, Medium, Hard).	Target for Classification Model.

Data Preprocessing Steps:-

Data preprocessing is the "cleanup" phase. We take raw, messy text and make it consistent.

A. Handling Missing Values

- Technique: Imputation with empty strings.
- Logic: Our dataset may often have missing fields. We used `data.fillna("")` to replace NaN values with empty strings.

B. Feature Concatenation

- Technique: String Concatenation.
- Logic: We created a new column `full_text` by merging three distinct columns:

$$\text{Full_Text} = \text{Description} + \text{Input_Description} + \text{Output_Description}$$

C. Custom Text Cleaning

- Technique: Regular Expressions .

- Logic:
 1. Lowercasing: Converted all text to lowercase to treat "Matrix" and "matrix" as the same word.
 2. HTML Removal: Used regex to strip HTML tags like <p>, <div>,
 which are common in web-scraped data.
 3. Selective Character Filtering:
 - Standard NLP removes all punctuation.
 - Our Approach: We explicitly preserved mathematical symbols (^, <, =, +, *, -) because in coding problems, "N < 1000" vs "N < 10^9" makes a huge difference in difficulty.

D. Label Encoding

- Technique: LabelEncoder from Scikit-Learn.
- Logic: The model cannot understand words like "Easy". We converted the target variable problem_class into integers:
 - Easy -> 0
 - Medium -> 1
 - Hard -> 2

Feature Engineering Techniques:-

Feature engineering is where we turn text into numbers (vectors) that have "meaning."

A. TF-IDF Vectorization

- Technique: Term Frequency - Inverse Document Frequency (TfidfVectorizer).
- Logic: Instead of just counting words , TF-IDF weighs them.
 - Common words (like "the", "write") get low scores.
 - Rare, specific words (like "recursion") get high scores.
- Configuration: We restricted the vocabulary to the top 5,000 most important features (max_features=5000) to keep the model fast and prevent overfitting.

B. N-Gram Analysis

- Technique: Bigrams (ngram_range = (1, 2)).
- Logic: We told the model to look at single words (Unigrams) AND pairs of words (Bigrams).
 - *Unigram*: "dynamic", "programming"
 - *Bigram*: "dynamic programming"

C. Stop Word Removal

- Technique: stop_words = 'english'.
- Logic: We removed common English filler words (is, am, are, the) so the model focuses purely on the technical content .

Models Used:-

We utilized a Dual-Model Architecture based on the XGBoost (Extreme Gradient Boosting) algorithm. We trained two

separate models to handle the two different tasks (Classification and Regression) simultaneously.

A. Classification Model (XGBClassifier)

- Purpose: To categorize the problem into discrete difficulty levels: Easy, Medium, Hard.
- Algorithm: XGBoost Classifier. It is an ensemble learning method that builds hundreds of decision trees sequentially, where each new tree corrects the errors of the previous ones.
- Key Hyperparameters:
 - `n_estimators=200`: The model builds 200 decision trees to ensure robustness.
 - `max_depth=6`: Limits the depth of each tree to prevent overfitting (learning noise instead of patterns).
 - `learning_rate=0.05`: Controls how much each tree contributes to the final prediction (lower is usually more accurate but slower).

B. Regression Model (XGBRegressor)

- Purpose: To predict the precise numerical difficulty score (e.g., 4, 6, 7, 8, ...).
- Algorithm: XGBoost Regressor. Instead of voting for a class, the trees in this model output numerical values which are averaged to get the final score.

Experimental Setup:-

Experimental Setup

A. Environment

- Language: Python
- Core Libraries:
 - Scikit-learn: For data splitting, label encoding, and TF-IDF vectorization.
 - XGBoost: For the machine learning algorithms.
 - Pandas: For data manipulation.
 - Joblib: For saving and loading the trained models (.pkl files).

B. Data Splitting

- Technique: Train-Test Split.
- Ratio: We used an 80/20 split
 - 80% of data: Used to train the models (teach them the patterns).
 - 20% of data: Held back as "unseen" data to test the models' accuracy

Evaluation Metrics:-

1. Classification Metrics (For Predicting Difficulty Level)

These metrics measure how well the model separates problems into Easy, Medium, and Hard.

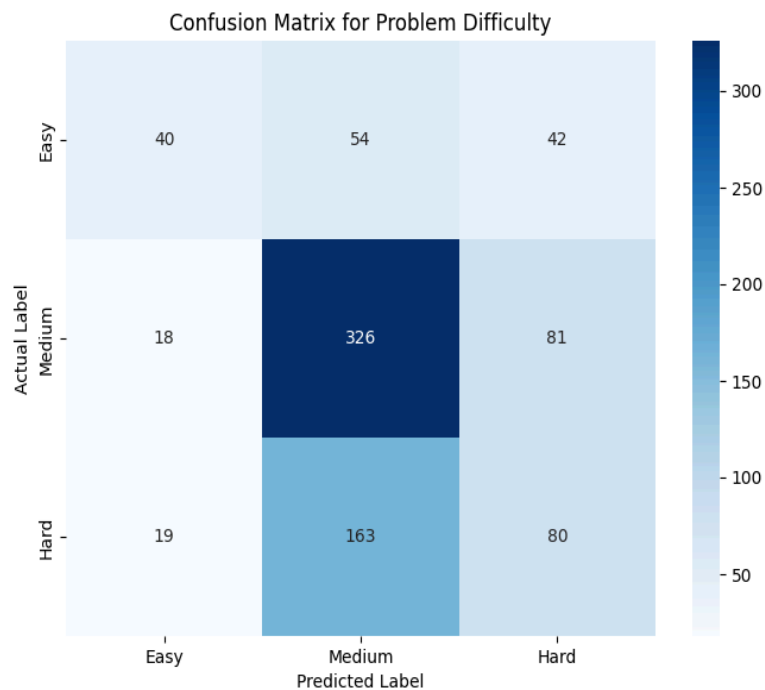
A. Accuracy

- This tells us that how accurately our model is predicting the problems.
- It is the most intuitive metric to explain to humans.
- The Accuracy of our AutoJudge project is 54.19%.

Accuracy: 54.19%

Detailed Report:

	precision	recall	f1-score	support
easy	0.52	0.29	0.38	136
hard	0.60	0.77	0.67	425
medium	0.39	0.31	0.34	262
accuracy			0.54	823
macro avg	0.50	0.46	0.46	823
weighted avg	0.52	0.54	0.52	823



2. Regression Metrics (For Predicting Score)

These metrics measure how close the predicted rating (e.g., 5.5) is to the actual rating (e.g., 5).

A. Mean Absolute Error (MAE)

- It is the average difference between the predicted score and the actual score.
- Our MAE for this project comes out to be 1.69/10

B. Root Mean Squared Error (RMSE)

- Similar to MAE, but it squares the error before averaging.

- Our RMSE for this project is 2.05/10.

```
4. Training XGBoost Regressor...  
✓ RMSE: 2.05/10  
✓ MAE: 1.69/10
```

Explanation of Web Interface:-

1. Technology Stack

The frontend is built using Next.js, a modern React-based framework.¹ This ensures the application is fast, responsive, and capable of handling dynamic state changes (like updating scores without reloading the page).²

- Styling: Tailwind CSS is used for a clean, modern aesthetic. It provides a responsive layout that works on different screen sizes.³
- State Management: React Hooks (useState) are used to manage the input text, loading status, and result data in real-time.⁴

2. User Workflow & Key Components

The interface is designed with a Single-Page Application (SPA) architecture, divided into three logical sections:

A. The Input Field

- Component: A large, multi-line <textarea> specifically designed to handle unstructured text. The User enters the problem description including inputs and outputs.

B. The Control Mechanism

- Component: A "Predict Difficulty" button.
- Functionality: When clicked, it triggers an asynchronous HTTP POST request to the Flask backend.
- UX Feedback: While the Python backend processes the heavy ML logic, the button enters a "Loading..." state to inform the user that the system is working, preventing duplicate submissions.

C. The Results Dashboard

Once the model returns a prediction, the interface dynamically renders a results card containing:

1. Difficulty Label: The predicted class (Easy / Medium / Hard), often color-coded (e.g., Green for Easy, Red for Hard) for instant visual recognition.
2. Predicted Score: The precise numerical score (e.g., "8") generated by the Regression model.
3. Estimated Codeforces Rating :- Added the feature of getting estimated codeforces rating range of a problem.

3. Integration Logic

The web interface acts as a decoupled client. It does not contain any ML logic itself.

1. User pastes text → Frontend cleans strings (trims whitespace).
2. Frontend sends JSON payload to `http://localhost:5000/predict`.
3. Frontend receives JSON response → Updates React State → UI re-renders automatically.

Paste the Problem Statement here (Description, Input, and Output details).

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.
Example:
Input: heights = [2,1,5,6,2,3]
Output: 10 (The rectangle formed by heights 5 and 6).

266 characters |

Predict Difficulty

DIFFICULTY LEVEL Medium	COMPLEXITY SCORE 6.7 /10
--	---

Estimated Codeforces Range: 1600 - 1899

Conclusion

The "AutoJudge" project successfully demonstrates that the difficulty of a programming problem can be predicted with reasonable accuracy solely based on its textual description. By integrating Natural Language Processing (Statistical NLP) with Machine Learning (XGBoost), we created a system that

removes the subjectivity and latency associated with manual difficulty rating